Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

Дисциплина: «ПиК ЯП»

Рубежный Контроль 1

Тема: «Предметная область: Операционная система - Компьютер (Вариант В9)»

Выполнил: студент гр. ИУ5-23Б _____ (Пенов М.С.)
(Подпись) (Ф.И.О.)

Проверил: _____ (Гапанюк Ю.Г)
(Подпись) (Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:
- Оценка:

2026 г.

1. Общее описание задания

Необходимо разработать программу на языке Python, выполняющую следующие действия:

1) Создать два класса данных в соответствии с вариантом предметной области (Операционная система - Компьютер, Вариант 9), которые связаны отношениями один-ко-многим и многие-ко-многим.

2) Создать списки объектов классов, содержащих тестовые данные, таким образом, чтобы первичные и вторичные ключи соответствующих записей были связаны по идентификаторам.

3) Разработать запросы в соответствии с вариантом "В". При разработке запросов необходимо использовать функциональные возможности языка Python (comprehensions, функции высших порядков).

2. Базовый код: структура классов и тестовые данные

```
from operator import itemgetter

# Класс 1: Операционная система (аналог "Сотрудник")
class OS:
    def __init__(self, id, name, version, comp_id):
        self.id = id
        self.name = name
        self.version = version
        self.comp_id = comp_id

# Класс 2: Компьютер (аналог "Отдел")
class Computer:
    def __init__(self, id, name):
        self.id = id
        self.name = name

# Класс для связи многие-ко-многим
class OSComp:
    def __init__(self, comp_id, os_id):
        self.comp_id = comp_id
        self.os_id = os_id

# Тестовые данные: Компьютеры
computers = [
    Computer(1, 'Acer Aspire 5'),
    Computer(2, 'Dell Latitude 5540'),
    Computer(3, 'HP ProBook 450'),
]

# Тестовые данные: Операционные системы
os_list = [
    OS(1, 'Android 14',     14.0,   1),
    OS(2, 'Arch Linux',     2024.0, 2),
    OS(3, 'Windows 11',     11.0,   1),
    OS(4, 'Ubuntu 22.04',   22.04,  3),
    OS(5, 'Astra Linux 1.7', 1.7,   3),
]
```

```
# Тестовые данные: Связи многие-ко-многим (ОС <-> Компьютер)
os_comps = [
    OSComp(1, 1),
    OSComp(2, 2),
    OSComp(1, 3),
    OSComp(3, 4),
    OSComp(3, 5),
]

# Формируем таблицу один-ко-многим
one_to_many = [
    (os.name, os.version, c.name)
    for c in computers
    for os in os_list
    if os.comp_id == c.id
]
```

3. Задачи и выполнение запросов

3.1. Запрос 1 (связь один-ко-многим)

Описание задачи: Выведите список всех операционных систем, у которых название начинается с буквы «А», и названия их компьютеров.

Текст программы:

```
res_1 = sorted(
    [(os_name, comp_name)
     for os_name, version, comp_name in one_to_many
     if os_name.startswith('A')],
    key=itemgetter(0)
)
print('\nЗапрос 1 — ОС с названием на "A" и их компьютеры:')
print(res_1)
```

Экранная форма с примером выполнения:

```
Запрос 1 — ОС с названием на "A" и их компьютеры:
[('Android 14', 'Acer Aspire 5'), ('Arch Linux', 'Dell Latitude 5540'), ('Astra Linux 1.7', 'HP ProBook 450')]
```

Рисунок 1 – Результат выполнения запроса 1

3.2. Запрос 2 (связь один-ко-многим)

Описание задачи: Выведите список компьютеров с минимальной версией ОС в каждом компьютере, отсортированный по минимальной версии.

Текст программы:

```
res_2_unsorted = []
for c in computers:
    # Фильтруем ОС данного компьютера из таблицы один-ко-многим
    c_os = list(filter(lambda i: i[2] == c.name, one_to_many))
    if len(c_os) > 0:
        # Извлекаем версии ОС данного компьютера
        c_versions = [version for _, version, _ in c_os]
        # Минимальная версия ОС в данном компьютере
        c_min_version = min(c_versions)
        res_2_unsorted.append((c.name, c_min_version))
```

```
res_2 = sorted(res_2_unsorted, key=itemgetter(1))
print('\nЗапрос 2 — Компьютеры с мин. версией ОС (сортировка по версии):')
print(res_2)
```

Экранная форма с примером выполнения:

```
Запрос 2 — Компьютеры с мин. версией ОС (сортировка по версии):
[('HP ProBook 450', 1.7), ('Acer Aspire 5', 11.0), ('Dell Latitude 5540', 2024.0)]
```

Рисунок 2 – Результат выполнения запроса 2

3.3. Запрос 3 (связь многие-ко-многим)

Описание задачи: Выведите список всех связанных операционных систем и компьютеров, отсортированный по операционным системам (сортировка по компьютерам произвольная).

Текст программы:

```
many_to_many_temp = [
    (c.name, oc.comp_id, oc.os_id)
    for c in computers
    for oc in os_comps
    if oc.comp_id == c.id
]

many_to_many = [
    (os.name, os.version, comp_name)
    for comp_name, comp_id, os_id in many_to_many_temp
    for os in os_list
    if os.id == os_id
]

res_3 = sorted(many_to_many, key=itemgetter(0))
print('\nЗапрос 3 — Все связанные ОС и компьютеры (мн-ко-мн, сортировка по ОС):')
print(res_3)
```

Экранная форма с примером выполнения:

```
Запрос 3 — Все связанные ОС и компьютеры (мн-ко-мн, сортировка по ОС):
[('Android 14', 14.0, 'Acer Aspire 5'), ('Arch Linux', 2024.0, 'Dell Latitude 5540'), ('Astra Linux 1.7', 1.7, 'HP ProBook 450'), ('Ubuntu 22.04', 22.04, 'HP
 ProBook 450'), ('Windows 11', 11.0, 'Acer Aspire 5')]
PS C:\Users\USER\VSCODE\PCPL>
```

Рисунок 3 – Результат выполнения запроса 3